

1. What is Blockchain?

1.1. Definition

Blockchain is a **distributed and decentralized digital ledger** that records transactions across many computers in such a way that the registered transactions **cannot be altered retroactively**.

In simpler terms:

A blockchain is a **chain of blocks**, where each block contains a list of transactions and is linked to the previous block using **cryptography**.

1.2 The Four Core Properties

1.2.1 Decentralization

- **Traditional System:** A central authority (like a bank) controls all data
- **Blockchain:** Data is distributed across many computers (nodes)
- No single point of failure
- No single entity has complete control

Example Analogy:

Traditional Banking:

[You] → [Bank Server] ← [Friend]
(Single point)

Blockchain:

[You] ↔ [Node1] ↔ [Node2] ↔ [Node3] ↔ [Friend]
(All nodes have the same information)

1.2.2. Immutability

- Once data is recorded, it cannot be changed or deleted
- Any attempt to modify previous records is immediately detectable
- Creates a permanent, tamper-proof history

Why it matters: Prevents fraud and ensures data integrity

1.2.3. Transparency

- All transactions are visible to all participants

- Anyone can verify the transaction history
- While transparent, participants can remain pseudonymous (identified by addresses, not real names)

Example: You can see that "Address A sent 5 coins to Address B" but you may not know who owns these addresses

1.2.4. Security

- Uses advanced cryptography to secure data
- Each transaction is digitally signed
- Consensus mechanisms prevent malicious actors from taking control
- Cryptographic hashing ensures data integrity

1.3. Basic Idea

Each block in a blockchain contains:

- **Block Header** – includes metadata such as timestamp and hash of the previous block.
- **Transaction Data** – actual list of transactions.
- **Hash Value** – a unique digital fingerprint for the block.

All blocks are connected in sequence — forming a “chain”.

Block 1 → Block 2 → Block 3 → Block 4 → ...

Each block’s **hash** depends on the previous block — so if someone tries to change Block 2, it breaks the entire chain.

1.4. Key Features of Blockchain

Feature	Description
Decentralization	No central authority — data is shared among all nodes.
Transparency	Every participant can view the transactions.
Immutability	Once data is added, it cannot be modified or deleted.
Security	Uses cryptography and consensus algorithms.

Feature	Description
Consensus Mechanism	Agreement among participants (e.g., Proof of Work, Proof of Stake).

1.5. Structure of a Block

Let's visualize it:

Block Header	
- Block Number	
- Timestamp	
- Previous Hash	
- Nonce	
- Hash	

Transactions:	
1. Alice → Bob : 2 BTC	
2. Bob → Charlie : 1 BTC	

1.6. How Blockchain Works (Step-by-Step)

Let's trace how a transaction moves through the blockchain system using **Alice sending money to Bob** as our example.

Step-by-Step Transaction Flow

Step 1: Transaction Creation

Alice wants to send 10 BTC to Bob	
Alice creates transaction:	
- From: Alice's address	
- To: Bob's address	
- Amount: 10 BTC	
- Signs with her private key	

Step 1: Transaction Initiation

- Alice decides to send 10 cryptocurrency units to Bob
- She creates a transaction using her wallet software
- Transaction details include:
 - Sender's address (Alice)

- Receiver's address (Bob)
- Amount to transfer
- Timestamp
- Transaction fee

Step 2: Digital Signing

- Alice signs the transaction with her **private key**
- This proves that Alice authorized this transaction
- Only Alice has her private key (like a digital signature)

```
# Simplified example of transaction signing (conceptual)
from hashlib import sha256
import json

# Transaction data
transaction = {
    "from": "Alice_Address_123",
    "to": "Bob_Address_456",
    "amount": 10,
    "timestamp": "2025-10-12 14:30:00"
}

# Create transaction hash
transaction_string = json.dumps(transaction, sort_keys=True)
transaction_hash = sha256(transaction_string.encode()).hexdigest()

print(f"Transaction Hash: {transaction_hash}")
# Output: Transaction Hash: a7f5c3d9e8b2f1a4c6e8d9f2b3a5c7e9...
```

Step 3: Broadcasting to Network

- The signed transaction is broadcast to all nodes in the network
- Nodes receive and validate the transaction:
 - Does Alice have sufficient balance?
 - Is the signature valid?
 - Is the transaction properly formatted?

Step 4: Transaction Pool (Mempool)

- Valid transactions wait in a "memory pool" (mempool)
- Miners/validators select transactions from this pool
- Transactions with higher fees are often prioritized

Step 5: Block Creation

- A miner/validator groups multiple transactions together
- These transactions form a new block
- The block includes:
 - Transaction data
 - Previous block's hash (creates the chain link)
 - Timestamp
 - Nonce (for Proof-of-Work)

Step 6: Consensus Mechanism

- Miners compete to solve a cryptographic puzzle (PoW) OR
- Validators are selected to propose the block (PoS)
- The winning miner/validator broadcasts their block

Step 7: Block Verification

- Other nodes verify the block:
 - Are all transactions valid?
 - Is the cryptographic puzzle solved correctly?
 - Does the previous block hash match?

Step 8: Block Addition

- If verified, the block is added to the blockchain
- All nodes update their copy of the blockchain
- Alice's transaction to Bob is now permanently recorded

Step 9: Confirmation

- Bob can now see the transaction
- Additional blocks built on top provide "confirmations"
- More confirmations = more security (transaction is deeper in the chain)

Visual Timeline:

[Transaction Created] → [Signed] → [Broadcast] → [Mempool] →
[Block Creation] → [Mining/Validation] → [Verification] →
[Added to Chain] → [Confirmed]

1.7. Example: Simple Blockchain in Python

Let's simulate a **very basic blockchain**.

```

import hashlib
import datetime

# Define a block structure
class Block:
    def __init__(self, index, previous_hash, data):
        self.index = index
        self.timestamp = str(datetime.datetime.now())
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        # Hash combines index, timestamp, data, and previous hash
        block_string = str(self.index) + self.timestamp + self.data + self.previous_hash
        return hashlib.sha256(block_string.encode()).hexdigest()

# Create the first block - Genesis Block
genesis_block = Block(0, "0", "Genesis Block")
print("Genesis Block Hash:", genesis_block.hash)

# Create the next block
block1 = Block(1, genesis_block.hash, "Ali sends 2 BTC to Majid")
print("Block 1 Hash:", block1.hash)

# Create another block
block2 = Block(2, block1.hash, "Sajid sends 1 BTC to Fahad")
print("Block 2 Hash:", block2.hash)

```

Output Example:

```

Genesis Block Hash: 8d7b4f...
Block 1 Hash: 14cd1a...
Block 2 Hash: 7e1a2b...

```

Each block contains the hash of the previous one — creating a secure link.

1.8. Types of Blockchains

Type	Description	Example
Public	Open to anyone. Anyone can join, read, or write.	Bitcoin, Ethereum
Private	Controlled by an organization. Only invited users can join.	Hyperledger Fabric

Type	Description	Example
Consortium (Hybrid)	Shared among selected organizations.	Banking systems, Government projects

1.9. Applications of Blockchain

Sector	Use Case
Finance	Cryptocurrencies (Bitcoin, Ethereum)
Healthcare	Secure patient records
Supply Chain	Product tracking and transparency
Voting Systems	Tamper-proof election systems
Education	Secure academic records and certifications

1.10. Limitations

- High energy consumption (Proof of Work)
- Slower transaction speed
- Data privacy issues in public blockchains
- Scalability challenges

2. The Byzantine Generals Problem

Before diving into consensus mechanisms, we must understand the fundamental problem they solve.

The Scenario

Imagine several Byzantine generals surrounding an enemy city. They must coordinate to attack simultaneously to win. However:

- Generals communicate via messengers
- Some generals might be traitors sending false messages
- Messages can be intercepted or delayed
- **Challenge:** How do loyal generals reach consensus on attack time despite traitors?

The Blockchain Parallel

Byzantine Generals Problem → Blockchain Problem

Generals	→ Network Nodes
Messengers	→ Network Communication
Traitors	→ Malicious Actors
Coordinated Attack	→ Agreed Blockchain State

Key Question: How can distributed nodes agree on transaction order and validity without trusting each other or a central authority?

Answer: Consensus Mechanisms (PoW, PoS, and others)

2.1. Nonce and Difficulty in Blockchain

2.1.1. What is Nonce?

The **Nonce** (short for *Number used once*) is a **random or sequential number** that miners keep changing until they find a valid hash for a block.

In simple words:

The **nonce** is like a “guess number” that miners change repeatedly to find a hash that meets the blockchain’s **difficulty requirement**.

Where it’s used

In the block’s data, the hash is calculated as:

```
Hash = SHA256(index + timestamp + data + previous_hash + nonce)
```

If the resulting hash doesn’t satisfy the rule (like starting with 3 zeros), the miner **increases the nonce** and **tries again**.

Example

Imagine we need a hash that starts with “000”.

```
import hashlib

data = "Alice sends 2 BTC to Bob"
nonce = 0
```



```
while True:
    hash_result = hashlib.sha256((data + str(nonce)).encode()).hexdigest()
    if hash_result.startswith("000"):
        print("✅ Valid Hash Found:", hash_result)
        print("Nonce used:", nonce)
        break
    nonce += 1
```

Example Output:

```
✅ Valid Hash Found: 000f12c8a9e13cfe9b3d...
Nonce used: 4589
```

Explanation:

- The computer keeps trying different nonces until it finds a hash that starts with “000”.
- Once found, the block is **mined successfully**.
- Nonce ensures randomness and makes each block unique.

2.2. What is Difficulty?

Difficulty is a number that determines **how hard it is to mine a block** — in other words, **how many leading zeros** the hash must have.

In simple words:

Difficulty tells miners how specific (rare) the target hash should be.

For example:

Difficulty	Required Hash Pattern
2	00xxxxxxxxxx...
3	000xxxxxxxxxx...
4	0000xxxxxxxxxx...

The higher the difficulty,
more leading zeros are required
more attempts (nonces) needed
longer mining time

2.2.1.Relationship Between Difficulty and Nonce

Concept	Description
Nonce	The variable that keeps changing to find a valid hash.
Difficulty	The rule that decides how “hard” it is to find the valid hash.
Mining Process	Keep changing the nonce until hash satisfies difficulty condition.

Example with Difficulty Change

Let's modify our previous example:

```
import hashlib, time

def mine_block(data, difficulty):
    nonce = 0
    prefix = "0" * difficulty
    start = time.time()

    while True:
        hash_result = hashlib.sha256((data + str(nonce)).encode()).hexdigest()
        if hash_result.startswith(prefix):
            end = time.time()
            print(f"Mined! Difficulty={difficulty}, Nonce={nonce}, Time={end-start}")
            print("Hash:", hash_result)
            break
        nonce += 1

mine_block("Ali pays Majid 2 BTC", difficulty=3)
mine_block("Ali pays Majid 2 BTC", difficulty=4)
```

Example Output:

```
Mined! Difficulty=3, Nonce=4789, Time=1.5s
Hash: 000a21c6d5f...
```

```
Mined! Difficulty=4, Nonce=52639, Time=15.2s
Hash: 00005dfc9b...
```

Explanation:

- When **difficulty = 3**, only 3 leading zeros are needed — found quickly.
- When **difficulty = 4**, it takes **much longer** — because valid hashes are rarer.

3. Proof-of-Work (PoW)

Concept: Miners compete to solve a computationally difficult mathematical puzzle. The first to solve it gets to add the next block.

How It Works:

1. Miners gather transactions into a block
2. They must find a nonce (number) that, when hashed with the block data, produces a hash with a specific pattern (e.g., starts with four zeros)
3. This requires billions of attempts (trial and error)
4. First miner to find the solution broadcasts the block
5. Other nodes verify the solution and add the block

Python Example of PoW Mining:

```
import hashlib
import time

class ProofOfWork:
    def __init__(self, block_data, difficulty=4):
        self.block_data = block_data
        self.difficulty = difficulty # Number of leading zeros require
        self.target = "0" * difficulty

    def mine(self):
        """Find a nonce that produces a hash with required difficulty"""
        nonce = 0
        start_time = time.time()

        while True:
            # Combine block data with nonce
            hash_input = f"{self.block_data}{nonce}"
            hash_result = hashlib.sha256(hash_input.encode()).hexdigest()

            # Check if hash meets difficulty requirement
            if hash_result.startswith(self.target):
                end_time = time.time()
                return nonce, hash_result, end_time - start_time

            nonce += 1

        # Show progress every 100,000 attempts
        if nonce % 100000 == 0:
            print(f"Attempt {nonce}... Current hash: {hash_result[:100]}")

# Example: Mining a block
```

```

block_data = "Alice → Bob: 10 BTC | Timestamp: 2025-10-12"
pow = ProofOfWork(block_data, difficulty=4)

print("Mining started...")
nonce, final_hash, time_taken = pow.mine()

print(f"\n✓ Block mined!")
print(f"Nonce found: {nonce}")
print(f"Final hash: {final_hash}")
print(f"Time taken: {time_taken:.2f} seconds")
print(f"Hash starts with {pow.difficulty} zeros: {final_hash.startswith('0' * pow.difficulty)}")

```

Advantages of PoW:

- ✓ Highly secure (requires massive computational power to attack)
- ✓ Battle-tested (Bitcoin has used it since 2009)
- ✓ Truly decentralized (anyone with hardware can participate)

Disadvantages of PoW:

- ✗ Extremely energy-intensive (high electricity consumption)
- ✗ Slow transaction processing
- ✗ Expensive hardware required
- ✗ Environmental concerns

4. Proof-of-Stake (PoS)

Concept: Validators are chosen to create new blocks based on how many coins they "stake" (lock up) in the network. No computational puzzle solving required.

How It Works:

1. Validators lock up (stake) a certain amount of cryptocurrency
2. The network randomly selects a validator to propose the next block
3. Selection probability often based on stake size (more stake = higher chance)
4. Other validators verify the proposed block
5. If accepted, the validator receives transaction fees as reward
6. Malicious behavior results in stake being "slashed" (taken away)

Simplified PoS Simulation:

```

import random

class ProofOfStake:

```

```

def __init__(self):
    self.validators = {}
    self.total_stake = 0

def add_validator(self, name, stake):
    """Add a validator with their stake"""
    self.validators[name] = stake
    self.total_stake += stake
    print(f"✓ {name} joined with stake: {stake} coins")

def select_validator(self):
    """Select validator based on stake (weighted random)"""
    # Create a list where each validator appears proportional to st
    selection_pool = []
    for validator, stake in self.validators.items():
        # Add validator multiple times based on stake
        selection_pool.extend([validator] * stake)

    # Randomly select from pool
    selected = random.choice(selection_pool)
    return selected

def simulate_block_creation(self, num_blocks=10):
    """Simulate multiple block creations"""
    print(f"\n--- Simulating {num_blocks} block creations ---")
    selection_count = {v: 0 for v in self.validators}

    for i in range(num_blocks):
        validator = self.select_validator()
        selection_count[validator] += 1
        print(f"Block {i+1}: Selected {validator}")

    print("\n--- Selection Statistics ---")
    for validator, count in selection_count.items():
        percentage = (count / num_blocks) * 100
        stake_percentage = (self.validators[validator] / self.total
        print(f"{validator}: {count} blocks ({percentage:.1f}%) | S

# Example simulation
pos = ProofOfStake()
pos.add_validator("Alice", 100)
pos.add_validator("Bob", 200)
pos.add_validator("Carol", 50)
pos.add_validator("Dave", 150)

pos.simulate_block_creation(100)

```

Advantages of PoS:

- ✓ Energy-efficient (no computational puzzles)
- ✓ Faster transaction processing
- ✓ Lower barrier to entry (no expensive hardware)
- ✓ Economic penalties discourage malicious behavior

Disadvantages of PoS:

- ✗ "Rich get richer" problem (more stake = more rewards)
- ✗ Less battle-tested than PoW
- ✗ Potential centralization if few hold most stake
- ✗ "Nothing at stake" problem (theoretical attack vector)

Comparison Table

Feature	Proof-of-Work (PoW)	Proof-of-Stake (PoS)
Energy Consumption	Very High (⚡⚡⚡⚡⚡)	Low (⚡)
Hardware Required	Specialized (ASICs)	Standard computers
Security Model	Computational power	Economic stake
Speed	Slower (10 min/block for Bitcoin)	Faster (seconds)
Decentralization	High	Moderate-High
Environmental Impact	Significant	Minimal
Examples	Bitcoin, Ethereum (old)	Ethereum (new), Cardano
Attack Cost	51% of network's hash power	51% of staked coins

5. Smart Contracts

5.1. Introduction

A **smart contract** is a **self-executing digital agreement** with the terms of the contract **directly written into code**.

It runs on a **blockchain network** (like Ethereum), ensuring that **agreements are automatically enforced** without the need for intermediaries such as lawyers, banks, or brokers.

Definition:

A **Smart Contract** is a program stored on a blockchain that runs automatically when predetermined conditions are met.

5.2. How Smart Contracts Work

Let's break it down step-by-step:

1. Agreement Creation

Two or more parties agree on rules or conditions.

2. Contract Written in Code

These rules are written as code (usually in **Solidity**, for Ethereum).

3. Deployment

The code is deployed to the blockchain network.

4. Execution

When the predefined conditions are met, the contract executes automatically.

5. Verification

Every node in the blockchain verifies the transaction.

6. Immutability

Once deployed, a smart contract **cannot be changed**.

Example: Real-World Analogy

Imagine a **vending machine**:

- You insert money → the machine checks the amount.
- If correct → it automatically releases the snack.
- If not → it returns your money.

No shopkeeper, no negotiation — just **automatic, rule-based execution**.

That's exactly how a **smart contract** works.

5.3. Structure of a Smart Contract

Typical components include:

Component	Description
Parties	The participants involved in the agreement.
Conditions	The logic or rules coded in the contract.
Trigger	An event or input that activates the contract.
Outcome	What happens when the condition is met or violated.

5.4. Example in Solidity (Ethereum)

Let's look at a simple **smart contract** written in Solidity that handles payments.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimplePayment {
    address public payer;
    address public payee;
    uint public amount;

    constructor(address _payee) payable {
        payer = msg.sender;          // Person deploying contract
        payee = _payee;              // Receiver of funds
        amount = msg.value;          // Amount sent with contract
    }

    // Function to release payment
    function releasePayment() public {
        require(msg.sender == payer, "Only payer can release payment");
        payable(payee).transfer(amount);
    }
}
```

Explanation:

- `constructor()` — runs once when contract is created.
- `msg.sender` — stores the address of the person deploying the contract.
- `msg.value` — stores the amount of cryptocurrency sent.
- `require()` — ensures that only authorized users execute the contract.
- `transfer()` — sends Ether from contract to the payee.

This is a **trustless** transaction — no need for a bank to verify payment.

5.5. Lifecycle of a Smart Contract

Write Code → Deploy on Blockchain → Wait for Trigger → Execute Automatically

Example:

- Buyer and seller agree to sell a car.
- Smart contract: “If buyer sends 10 ETH, then transfer car ownership.”
- When condition met → ownership changes automatically.

5.6. Smart Contract Platforms

Platform	Language	Key Feature
Ethereum	Solidity	First and most popular platform
Hyperledger Fabric	Go / JavaScript	Permissioned blockchain
EOS	C++	High performance and scalability
Cardano	Haskell	Research-based blockchain
NEO	Python / C#	Digital identity integration

5.7. Benefits of Smart Contracts

Benefit	Description
Automation	Executes automatically when conditions met
Trust	No third party — blockchain verifies everything
Transparency	Everyone can view contract logic and outcome
Security	Encrypted and immutable
Efficiency	Saves time and reduces paperwork
Cost Reduction	No intermediary fees

5.8. Limitations

Limitation	Description
Immutable Code	Bugs cannot be fixed after deployment

Limitation	Description
Complexity	Requires technical knowledge to code
Legal Recognition	Many countries don't have clear laws yet
External Data Dependency	Needs Oracles to get real-world information

5.9. Use Cases of Smart Contracts

Domain	Example
Finance (DeFi)	Automatic loan repayment or insurance claims
Real Estate	Automatic property transfer when payment confirmed
Healthcare	Secure patient data sharing
Supply Chain	Tracking goods automatically across logistics
Voting Systems	Transparent, tamper-proof elections
NFTs	Automatic ownership verification for digital assets

5.10. Example Use Case: Escrow Contract

Imagine an **online freelancer** who builds a website for a client.

Smart contract logic:

```

IF (work submitted == approved) THEN
    release payment to freelancer
ELSE
    refund client

```

- Funds are held securely in the blockchain.
- Execution is automatic.
- Neither party can cheat.

Lab Activity 01 — Understanding the Structure of a Block

Concept

Each block in a blockchain has:

1. **Index** – block number.
2. **Timestamp** – time of creation.
3. **Data** – transaction information.
4. **Previous Hash** – link to previous block.
5. **Hash** – unique fingerprint of the block.

Example Code

```
import hashlib
import datetime

# Define Block class
class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    # Function to calculate hash of the block
    def calculate_hash(self):
        block_string = str(self.index) + str(self.timestamp) + str(self.data)
        return hashlib.sha256(block_string.encode()).hexdigest()

# Example block
block1 = Block(1, str(datetime.datetime.now()), "Ali sends 2 BTC to Sajjad")

print("Block Index:", block1.index)
print("Timestamp:", block1.timestamp)
print("Data:", block1.data)
print("Previous Hash:", block1.previous_hash)
print("Hash:", block1.hash)
```

Output Example

```
Block Index: 1
Timestamp: 2025-10-10 14:25:18.256743
Data: Ali sends 2 BTC to Sajjad
Previous Hash: 0
Hash: e3a1b2c5c...
```

Explanation:

- The block generates a **SHA-256 hash** based on its contents.
- Any small change in data produces a completely new hash.

Lab Activity 02 — Creating a Simple Blockchain

Concept

A **Blockchain** is a chain of linked blocks.

Each new block's `previous_hash` is set to the hash of the block before it.

Code Example

```
import hashlib
import datetime

# Define a Block class
class Block:
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        block_string = str(self.index) + str(self.timestamp) + str(self.data)
        return hashlib.sha256(block_string.encode()).hexdigest()

# Define a Blockchain class
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, str(datetime.datetime.now()), "Genesis Block", "")

    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, new_data):
        latest_block = self.get_latest_block()
        new_block = Block(len(self.chain), str(datetime.datetime.now()), new_data, latest_block.hash)
        self.chain.append(new_block)
```

```
# Create Blockchain
my_chain = Blockchain()
my_chain.add_block("Ali sends 2 BTC to Sajjad")
my_chain.add_block("Noor sends 1 BTC to Umair")

# Display blocks
for block in my_chain.chain:
    print(f"Block {block.index} => Hash: {block.hash}")
    print(f"Previous Hash: {block.previous_hash}\n")
```

Output Example

```
Block 0 => Hash: 8a41c3d...
Previous Hash: 0
```

```
Block 1 => Hash: c9a3e7a...
Previous Hash: 8a41c3d...
```

```
Block 2 => Hash: 1ef42aa...
Previous Hash: c9a3e7a...
```

Explanation:

- The blockchain starts with a **genesis block**.
- Each new block stores the hash of the previous block, forming a **linked chain**.

Lab Activity 03 — Validating Blockchain Integrity

Concept

If anyone tries to **modify** data in any block, the chain should become **invalid**. We verify the chain by checking:

1. Each block's stored hash is correct.
2. Each block's `previous_hash` matches the previous block's actual hash.

Code Example

```
def is_chain_valid(blockchain):
    for i in range(1, len(blockchain.chain)):
        current = blockchain.chain[i]
```

```

previous = blockchain.chain[i-1]

# Verify hash integrity
if current.hash != current.calculate_hash():
    print(f"Block {i} hash mismatch!")
    return False

# Verify previous hash linkage
if current.previous_hash != previous.hash:
    print(f"Block {i} previous hash mismatch!")
    return False

print("Blockchain is valid.")
return True

# Check validity before and after tampering
print("\nChecking Blockchain Integrity:")
is_chain_valid(my_chain)

# Tamper with a block
my_chain.chain[1].data = "Iqbal sends 100 BTC to Umair" # changed data
print("\nAfter Tampering:")
is_chain_valid(my_chain)

```

Output Example

```

Checking Blockchain Integrity:
Blockchain is valid.

```

```

After Tampering:
Block 1 hash mismatch!

```

Explanation:

- Changing even a single character in the data **changes the hash**.
- The chain detects tampering automatically — ensuring **integrity**.

Lab Activity 04 — Mining Blocks (Proof of Work)

Concept

In real blockchains like **Bitcoin**, adding a block requires solving a computational puzzle — this is called **Proof of Work (PoW)**.

A miner must find a hash that starts with a certain number of zeros.

For example:

Hash must start with '0000'

This requires trying many different `nonce` values until success.

Code Example

```
import hashlib
import datetime

class Block:
    def __init__(self, index, timestamp, data, previous_hash, difficulty):
        self.index = index
        self.timestamp = timestamp
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.mine_block(difficulty)

    def calculate_hash(self):
        block_string = str(self.index) + str(self.timestamp) + str(self.data)
        return hashlib.sha256(block_string.encode()).hexdigest()

    def mine_block(self, difficulty):
        prefix = "0" * difficulty
        while True:
            new_hash = self.calculate_hash()
            if new_hash.startswith(prefix):
                print(f"Block mined: {new_hash}")
                return new_hash
            else:
                self.nonce += 1

class Blockchain:
    def __init__(self, difficulty=3):
        self.chain = [self.create_genesis_block()]
        self.difficulty = difficulty

    def create_genesis_block(self):
        return Block(0, str(datetime.datetime.now()), "Genesis Block", "", self.difficulty)

    def get_latest_block(self):
        return self.chain[-1]
```

```

def add_block(self, data):
    latest_block = self.get_latest_block()
    new_block = Block(len(self.chain), str(datetime.datetime.now()))
    self.chain.append(new_block)

# Test Mining
print("Mining Blockchain... Please Wait")
mine_chain = Blockchain()
mine_chain.add_block("Transaction: Qasim pays 5 BTC to Khalid")
mine_chain.add_block("Transaction: Majid pays 2 BTC to Saad")

```

Output Example

```

Mining Blockchain... Please Wait ⚡
✅ Block mined: 000a9f3b9cd45a3f92...
✅ Block mined: 0005b2f9de612345ab...

```

Explanation:

- `nonce` is incremented until the hash starts with required zeros.
- This simulates **mining effort**.
- Difficulty controls how hard it is to find the hash.

Complete Python Program – Blockchain Lab

```

import hashlib
import datetime

# -----
# BLOCK CLASS
# -----
class Block:
    def __init__(self, index, timestamp, data, previous_hash, difficult
        self.index = index                # Block number
        self.timestamp = timestamp        # Time of creation
        self.data = data                  # Transaction or record data
        self.previous_hash = previous_hash # Hash of previous block
        self.nonce = 0                    # Value changed to find valid
        self.hash = self.mine_block(difficulty) # Start mining on crea

    # Function to calculate the hash of the block
    def calculate_hash(self):
        block_string = (

```



```

        str(self.index)
        + str(self.timestamp)
        + str(self.data)
        + str(self.previous_hash)
        + str(self.nonce)
    )
    return hashlib.sha256(block_string.encode()).hexdigest()

# Mining process (Proof of Work)
def mine_block(self, difficulty):
    prefix = "0" * difficulty # Hash must start with N zeros
    while True:
        new_hash = self.calculate_hash()
        if new_hash.startswith(prefix):
            print(f"Block {self.index} mined with hash: {new_hash}")
            return new_hash
        else:
            self.nonce += 1 # Keep changing nonce until correct ha

# -----
# BLOCKCHAIN CLASS
# -----
class Blockchain:
    def __init__(self, difficulty=3):
        self.chain = [self.create_genesis_block()] # Start with Genesis
        self.difficulty = difficulty

    # Create the very first block (Genesis block)
    def create_genesis_block(self):
        print("Creating Genesis Block...")
        return Block(0, str(datetime.datetime.now()), "Genesis Block",

    # Get the most recent block in the chain
    def get_latest_block(self):
        return self.chain[-1]

    # Add new block to the chain
    def add_block(self, data):
        print(f"\nAdding new block with data: {data}")
        latest_block = self.get_latest_block()
        new_block = Block(
            len(self.chain),
            str(datetime.datetime.now()),
            data,
            latest_block.hash,
            self.difficulty,
        )
        self.chain.append(new_block)

```

```

# Validate the integrity of the blockchain
def is_chain_valid(self):
    print("\nChecking Blockchain Integrity...")
    for i in range(1, len(self.chain)):
        current = self.chain[i]
        previous = self.chain[i - 1]

        # Check if current block hash is still valid
        if current.hash != current.calculate_hash():
            print(f"Block {i} hash mismatch! Blockchain compromised")
            return False

        # Check if previous hash is correct
        if current.previous_hash != previous.hash:
            print(f"Block {i} previous hash mismatch! Blockchain broken")
            return False

    print("Blockchain is valid and secure.")
    return True

# Display the full blockchain
def display_chain(self):
    print("\nBlockchain Contents:")
    for block in self.chain:
        print(f"Block {block.index}")
        print(f"Timestamp      : {block.timestamp}")
        print(f>Data              : {block.data}")
        print(f"Nonce             : {block.nonce}")
        print(f"Previous Hash     : {block.previous_hash}")
        print(f"Hash              : {block.hash}\n")

# -----
# MAIN PROGRAM EXECUTION
# -----
if __name__ == "__main__":
    print("Starting Blockchain Simulation\n")

    # Initialize blockchain with difficulty level
    my_chain = Blockchain(difficulty=3)

    # Add sample transactions
    my_chain.add_block("Alice sends 2 BTC to Bob")
    my_chain.add_block("Bob sends 1 BTC to Charlie")
    my_chain.add_block("Charlie sends 0.5 BTC to David")

    # Display entire blockchain
    my_chain.display_chain()

```

```
# Validate blockchain
my_chain.is_chain_valid()

# Tampering example
print("\n⚠️ Tampering with block data...")
my_chain.chain[1].data = "Alice sends 200 BTC to Bob (Tampered)"
my_chain.is_chain_valid()
```

Explanation of the Complete Program

Section	Description
Block Class	Represents each block in the chain. Contains transaction data, timestamps, and hash generation.
mine_block()	Simulates Proof of Work — keeps changing <code>nonce</code> until the hash starts with N zeros.
Blockchain Class	Manages the chain, adds new blocks, and validates integrity.
Genesis Block	The first block in the chain, with <code>previous_hash = 0</code> .
Validation	Compares stored hash and recalculated hash to detect tampering.
Tampering Demo	Modifies data in one block to show that blockchain integrity breaks.

Sample Output

```
Starting Blockchain Simulation

Creating Genesis Block...
Block 0 mined with hash: 000b6a97d2...

Adding new block with data: Alice sends 2 BTC to Bob
Block 1 mined with hash: 0002fe89a1...

Adding new block with data: Bob sends 1 BTC to Charlie
Block 2 mined with hash: 0007b1cd44...

Adding new block with data: Charlie sends 0.5 BTC to David
Block 3 mined with hash: 0004c3de19...

Blockchain Contents:
Block 0
```

Timestamp : 2025-10-10 14:52:13.812392
Data : Genesis Block
Nonce : 256
Previous Hash : 0
Hash : 000b6a97d2...

Block 1

Timestamp : 2025-10-10 14:52:20.992100
Data : Alice sends 2 BTC to Bob
Nonce : 532
Previous Hash : 000b6a97d2...
Hash : 0002fe89a1...

Block 2

Timestamp : 2025-10-10 14:52:33.182349
Data : Bob sends 1 BTC to Charlie
Nonce : 786
Previous Hash : 0002fe89a1...
Hash : 0007b1cd44...

Block 3

Timestamp : 2025-10-10 14:52:45.103249
Data : Charlie sends 0.5 BTC to David
Nonce : 904
Previous Hash : 0007b1cd44...
Hash : 0004c3de19...

Checking Blockchain Integrity...
Blockchain is valid and secure.

Tampering with block data...
Block 1 hash mismatch! Blockchain compromised.



Practice Exercise for Students

Modify the above code to:

1. Display how long mining takes for each block (using `time.time()`).
2. Change difficulty to 4 or 5 and observe how mining time increases.
3. Add a function to **recalculate hashes** if tampering occurs (simulating chain repair).